

Creating a Customized Release Workflow



Pr

Available only for **ENT**

Administrators of enterprises can create customized release and obsolescence workflows specific to the enterprise needs. Onshape provides a JSON file of our release and obsolescence workflows as a starting place for customizing the workflow to suit your company's needs. You can also use our JSON as a starting point for creating your own JSON file.

Bear in mind that the currently selected release and obsolescence workflow (chosen by the administrator) governs newly created release candidates through the selected process. If you change the workflow while a release is in-process, that release follows the workflow under which it was created. The newly selected workflow will govern only release candidates created after it was selected.

One more thing: Onshape customized workflows currently do not allow cyclical processes.

You can unpublish a workflow, to remove it from the active workflow list.

You can add as many workflows as you want to your workflow repertoire to select from. You can also replace an existing customized workflow, if necessary.

We recommend keeping all of your workflow definition files in one Onshape document for ease of access.

The customization process has these major steps, each explained in detail below:

1. Download the Onshape default workflows to your hard drive or other easily accessible location.
2. Create a new Onshape document and import the workflow files.
3. Optionally customize a workflow with FeatureScript to update item properties upon some release action.
4. Edit and publish the workflows.
5. Optionally debug a customized workflow.

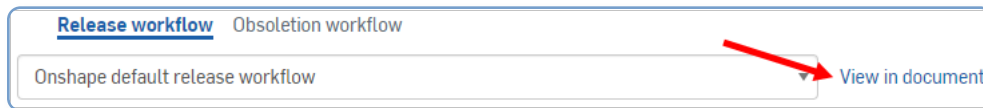
Automatic transition for two-tiered workflows

When you set up a custom release workflow that includes two tiers, and the second tier is not required (that is, the second tier is optional) Onshape will automatically transition from Pending to Released if there is no approver specified in the release package. No setup is required for this, the only requirements are that you have a two-tier workflow, the second tier is optional and there are no approvers specified in the Approver field in the release candidate.

Download the Onshape default workflows

To download the Onshape default workflow files:

1. Navigate to your [Enterprise settings](#) and select [Release management](#).
2. Confirm that Enable managed workflows is checked, then click View in document.



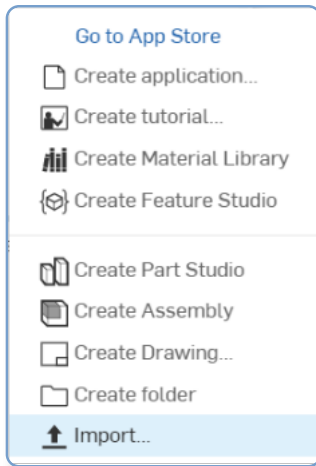
The workflow file opens in a public document in another tab.

3. Click Download, enter a new name for the file.

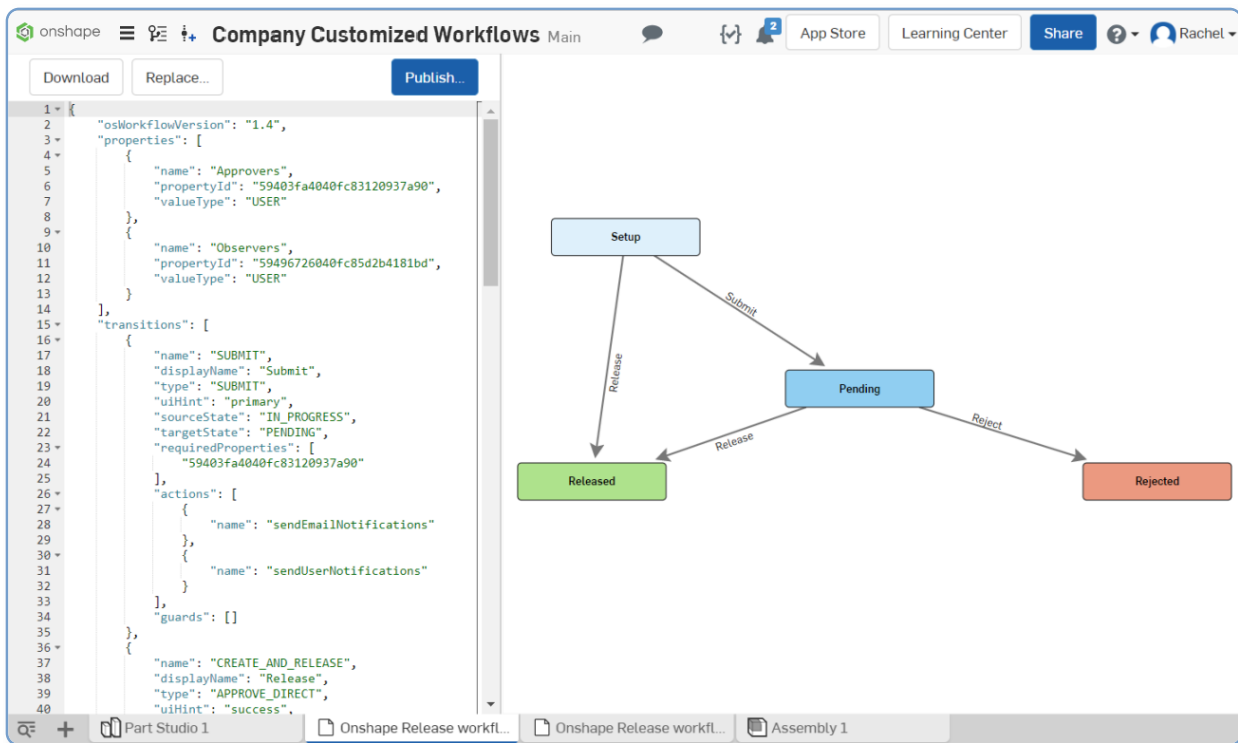
You can close that tab once the tab is downloaded.

Create a new Onshape document and import the workflow files

1. In your enterprise, navigate to the Documents page, select Create > Document, then select **OK**. (In this example, the document name is *Company Customized Workflows*)
2. In the new document, click the Plus icon  at the bottom and select Import:



3. Select the workflow file you just downloaded and click Open.
4. Another tab appears in your document, with the name of the file you imported. Select that tab to open it:



Customize with FeatureScript

You can use FeatureScript workflow actions to programmatically update properties on Release items as part of a workflow transition. By pointing the customized workflow at a Feature Studio in the same workspace as the workflow definition, a function in that Feature Studio can be executed as an action which determines the property updates to apply to each Release item.

To use this in a custom workflow, use the `updateItemPropertiesFromFeatureScript` workflow action as a transition action, or an an entry or exit action on a state. The action takes the parameters:

- **FeatureStudio ID** - The Feature Studio must reside in the same workspace as the customized release workflow definition
- **A function name exported by that Feature Studio** - This function must take a single argument that has type: map. The Map type passed into the function when it is executed contains this data:

```
{
// The full detailed release package, including all items and properties
```

```
// (as it would be returned by the getReleasePackage REST API)
"releasePackage": BTReleasePackageInfo,

// The user performing the transition
"currentUser": BTUserSummaryInfo,

// The user who originally created the Release candidate
"releaseCreator": BTUserSummaryInfo,

// The transition being executed, as it appears in the workflow definition
"transition": BTTransitionDef,

// The current UTC date/time, as an ISO date string
"currentDate": string
}
```

The function must return a map of the form:

```
{[key: string]: {[key: string]: any}}
```

The main map is keyed by Release item ID, and the inner map is from metadata property ID to a desired new value. For example, if the function returns the following map:

```
{
  "item1": {
    "57f3fb8efa3416c06701d616": "New value",
    "customIntProperty": 42
  },
  "item2": {
    "57f3fb8efa3416c06701d617": "Some other value"
  }
}
```

Then on the Release item with ID "item1", the "Title 1" property will be set to the string "New value", and a hypothetical custom property (customIntProperty, above) will be set to the number 42. On the item with ID "item2", the "Title 2" property will be set to the string "Some other value".

The full release package information is provided as input to the function, so additional filtering and computation can be done as desired. For example, applying a property only to items that are drawings, or only to items that have a certain category, or computing a property value from some other property value.



It is important to note that the FeatureScript itself does not apply any changes to the release package; any modifications made to the input data will not survive the FeatureScript execution. Instead, it is the value returned by the function that tells the system what changes to make.

Property value types

The value type in the returned map must match the value type of the corresponding metadata property (as it would be sent to the "update metadata" API); an invalid value will result in a failed property update.

A User-type metadata property accepts an array of objects of the form:

```
{"id": [userId]}
```

A User-type workflow property contains an array of objects where the user ID is in the "entryId" field. Therefore, to add a user from a workflow property to a metadata property, the value must be transformed to the correct format.

A User-type metadata property accepts only user IDs, not team or role IDs. To apply a User-type workflow property onto a User-type metadata property, use only entries in the workflow property that are users – those with an "entryType" of 0.

If any property updates fail due to a bad propertyID or value in the returned object, the workflow action will fail and the failures will be logged for workflow developers to use to debug (see [Workflow debugging](#), below).

The action also fails if an error occurs during FeatureScript execution or if the returned object is badly formed. For example, if the object returned is not a map of item ID to property updates.

If the action fails, it is likely because of a bug in the workflow FeatureScript rather than a bug in the system. In this case, you will receive an error message returned with the HTTP response directing you (or your users) to contact the workflow administrator. The workflow developer can use the workflow Audit log to identify and debug the failure (see [Workflow Audit logging](#), below). If necessary, the workflow developer can create a support ticket with Onshape.

Manage workflows global permission

This is a new global permission in Onshape Enterprise which allows publishing and debugging customized release workflows. Users with this permission can publish workflows, and will always be given the option to create a Release candidate in debug mode (see [Workflow debugging](#), below).

Edit and publish a workflow



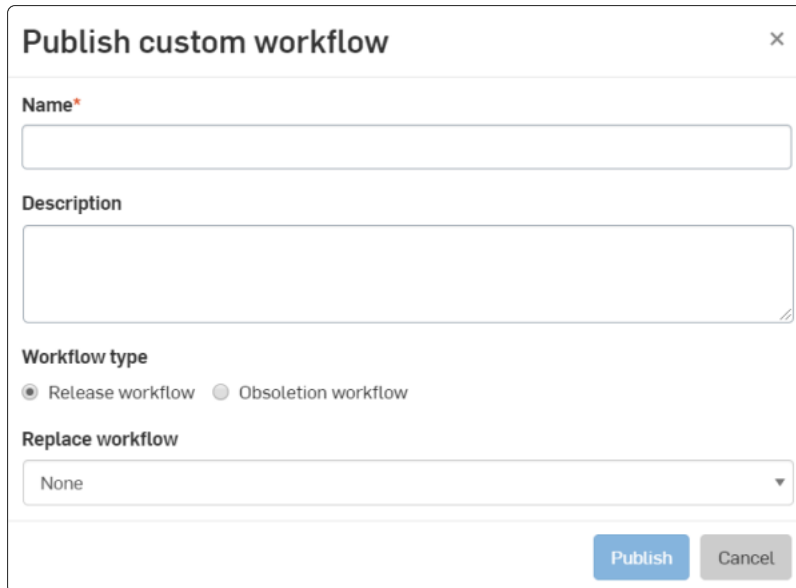
Cyclical workflows are not allowed.



Only users with the Manage Workflows global permission can publish workflows. For more information on Global permissions, see [Understanding Global Permissions](#).

Edit the custom workflow, as outlined below:

1. Make edits to the JSON workflow file. As you edit, the diagram on the right reflects the changes made.
2. The diagram is dynamic and draggable, so if the structure does not reflect your intended workflow, use your cursor to adjust it.
3. Once you are satisfied with your edits and want to make your workflow available for use in the enterprise, click **Publish** to open the Publish custom workflow dialog:



The dialog titled "Publish custom workflow" contains the following fields and controls:

- Name***: A text input field.
- Description**: A larger text input field.
- Workflow type**: Two radio buttons, "Release workflow" (selected) and "Obsolescence workflow".
- Replace workflow**: A dropdown menu currently showing "None".
- At the bottom right are two buttons: "Publish" (blue) and "Cancel" (grey).

The dialog alerts you of any errors that require fixing before publishing, displayed at the top of the dialog:

Publish custom workflow [X]

In transition CREATE_AND_RELEASE: Transition type must be one of SUBMIT, APPROVE, or REJECT

Name*

[Empty text field]

- a. Click **Cancel** to close the dialog.
 - b. Fix the errors.
 - c. Click **Publish** again to open the dialog.
4. Provide a Name, Description, and Workflow type.
 5. In the Replace workflow dropdown, do one of the following:
 - a. If this is a new workflow, select None.
 - b. To replace an already-existing workflow, select the one you wish to replace. The selected replacement Name and Description fields are automatically populated. You can keep the existing values or enter new ones:

Publish custom workflow [X]

Name*

01Review workflow

Description

Review workflow

Workflow type

☒ Release workflow ☐ Obsolescence workflow

Replace workflow

01Review workflow [v]

Publish **Cancel**

6. Click **Publish**.

Unpublishing a workflow

When a workflow is unpublished, the following occurs:

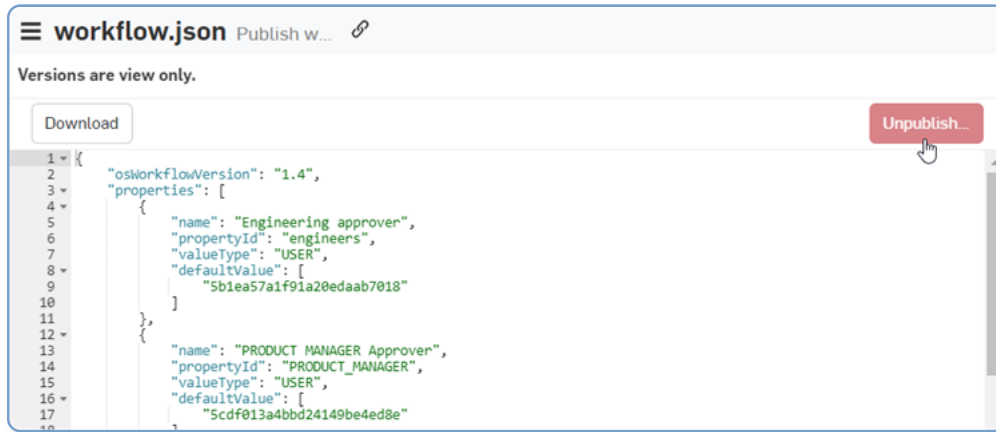
- The workflow is no longer available for use in Release management to create new release packages.
- Any settings using the removed workflow reset to the Onshape default workflow
- Release/Obsolescence candidates currently in the removed workflow continue to work normally. Existing release packages and submitted release packages using the unpublished workflow remain unaffected. You can still open, view, and obsolete existing release packages using the custom workflow and transition pending releases.

Only users who have Manage workflows permissions (under Enterprise settings > Global permissions) can unpublish a workflow.

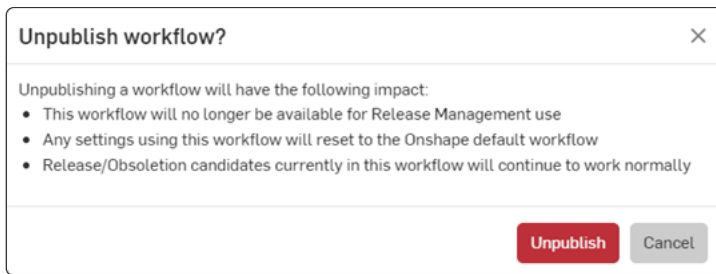
To unpublish a workflow:

1. Click your name or Account user icon () at the upper right on the Documents toolbar and select Enterprise settings > Release management.

2. Select the Release workflow to be unpublished from the workflow dropdown.
3. Click the View in document link to the right of the workflow dropdown. The workflow document opens in the current browser tab.
4. Click the `Unpublish` button at the top of the document.



5. The Unpublish workflow dialog opens. Click the `Unpublish` button.



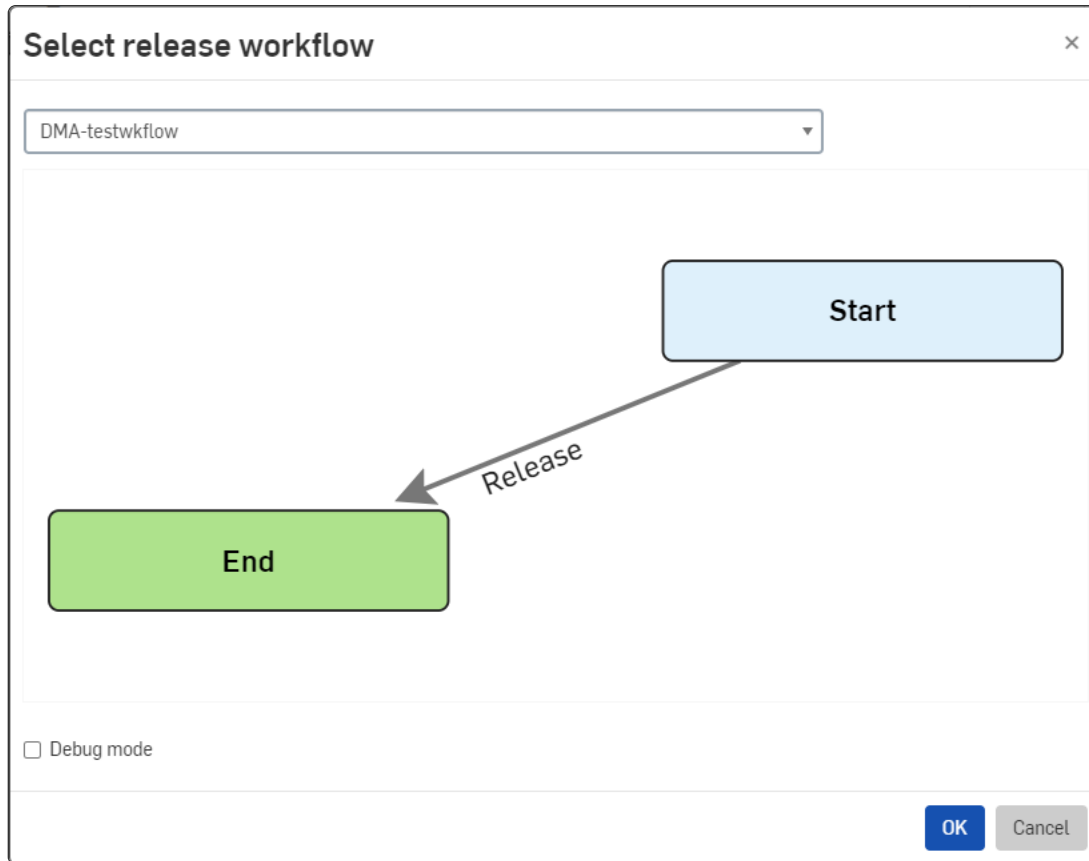
Unpublishing a default workflow resets the default workflow to the Onshape default.

Workflow debugging

A user with the Manage Workflows global permission can test a workflow in development using debug mode.



The workflow must first be published so it is available in the Select release workflow dialog upon release of a part/assembly. (Once you are satisfied with the new workflow, you can return to Enterprise settings and make the workflow active.)



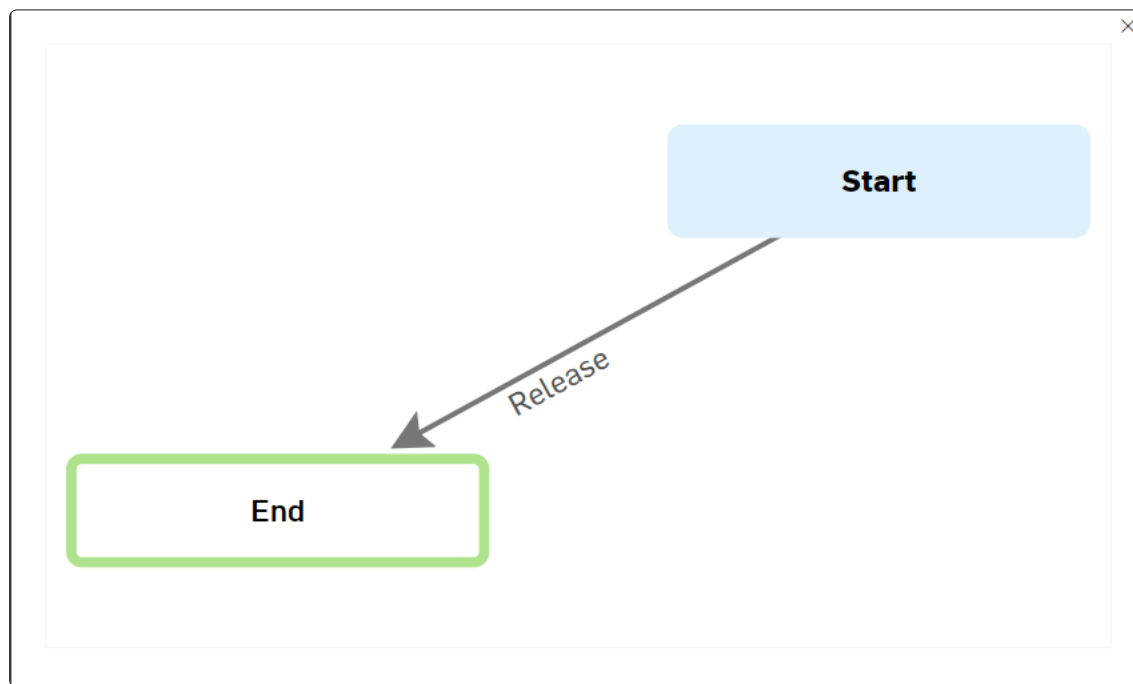
A typical published workflow points at a version of the document in which the workflow definition lives (the JSON file). Releases using that published workflow follow the workflow as defined in that version. Debug mode instead uses the workflow as it exists in the document workspace associated with the published version. Note that a workflow will only start in debug mode if there is exactly one workspace above the published version, thereby eliminating ambiguity.

Debug mode allows for fast iteration during workflow development. If you find that the workflow does not work as expected, you can make a change to the workflow JSON or the Feature Studio consumed by the new action in the workspace. Immediately starting a new release allows you (the workflow developer) to see the effects without needing to publish a new version of the workflow first.

To debug a workflow:

1. Check the Debug mode box (as seen in the image above).
2. Click **OK**. The Create Release candidate dialog opens:

3. In this dialog, you can select the overflow menu button (***) in the top right corner, then click [View audit log](#) to see an audit trail of the workflow objects and their significant events, for example, creation and transition.
4. You can also select View workflow to see the workflow as a diagram, with the last state highlighted, as in the diagram below:



Changes made in the workspace to a workflow in debug mode will not affect any release already in-progress. A debug release is locked to the microversion of the workspace at the time of creation. To see the results of the changes, you need to create a new release. This procedure prevents changes to the workflow definition from breaking an in-progress release and preventing it from being discarded.

Enabling multiple active workflows

Once you have published a custom workflow, you can enable multiple active workflows in your enterprise if you wish. When multiple active workflows are available to your users, they can select which workflow to follow when creating a release candidate.

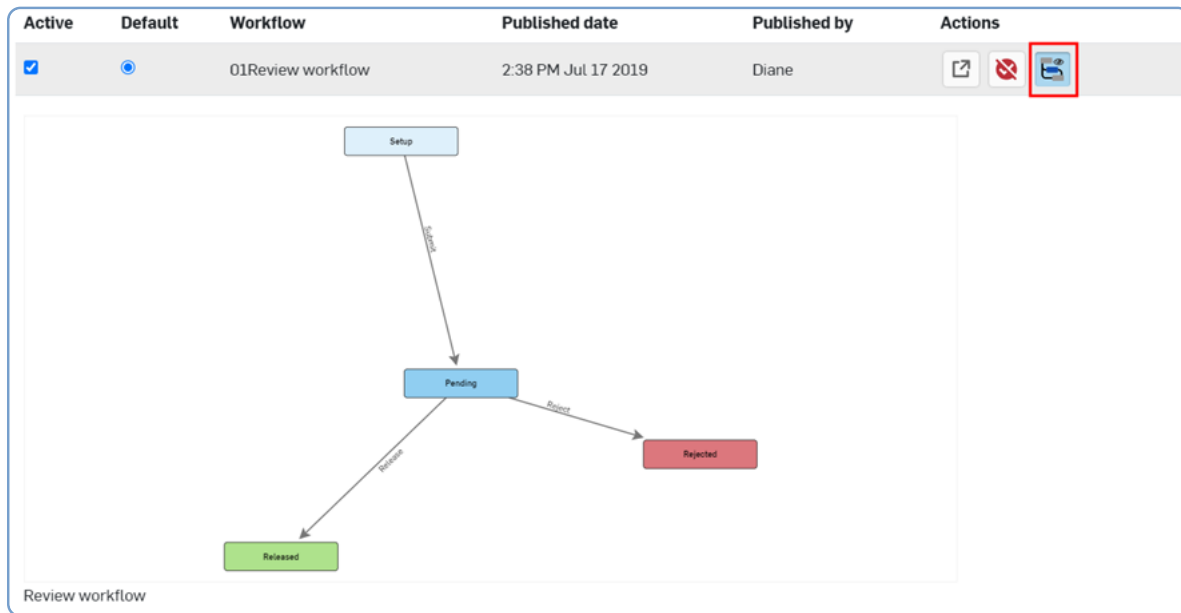
It is the responsibility of the administrator to educate users on which workflows should be used and under what conditions.

To enable multiple workflows:

1. Follow the instructions above to create more than one custom workflow and publish them in Onshape.
2. In the [Release management](#) page of [Enterprise settings](#), select which custom workflows to make active from the Managed workflows section by checking the Active box next to each workflow:

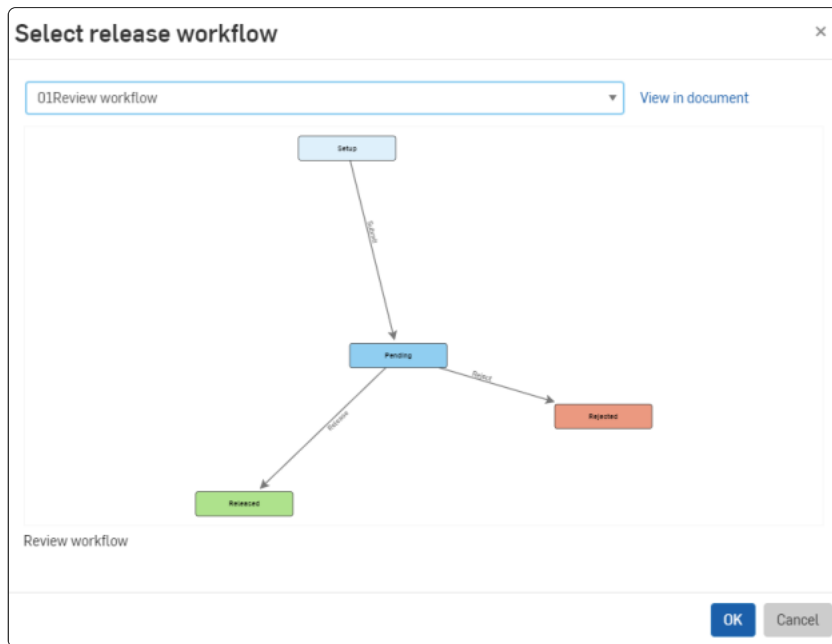
Workflows ?					
☒ Enable managed workflows					
Release workflow Obsolescence workflow					
☐ Onshape default workflows ☒ Custom workflows					
Active	Default	Workflow	Published date	Published by	Actions
☒	☒	01Review workflow	2:38 PM Jul 17 2019	Diane	  
☒	☐	02Supplier workflow	2:37 PM Jul 17 2019	Diane	  
☒	☐	03Production workflow	2:40 PM Jul 17 2019	Diane	  
☐	☐	zd-1	4:03 PM May 12 2021	Jason Manager	  
☐	☐	zen262806	3:40 PM Nov 1 2021	Jason Manager	  

3. Click the Workflow preview icon () to expand a dropdown of an illustration depicting the workflow. Beneath the illustration is a description of the current workflow process:



4. Make sure to click [Save release settings](#) to save the changes you've made to the Release management settings for the enterprise.

When your users create a release candidate, they are presented with an opportunity to select a workflow at that time (from the dropdown). If only one release workflow is available, this screen is only shown for admins and users with the [Manage workflows global permission](#).



Syntax

The syntax for customizing the release or obsolescence workflow is presented below. Note that capitalization is important.

Keep in mind the following rules.

Properties

Format

- name: string
- propertyId: string
- valueType: string|ENUM
- defaultValue?: any
- usersOnly?: boolean
- teamsOnly?: boolean
- enumValues?: list

Restrictions

- Property IDs must be unique (and not match Onshape's hard-coded Name, Description, or Comment property ID)
- valueType must be the name of a BTMetadataValueType (excluding BLOB and OBJECT)
- If valueType is ENUM, enumValues must be specified, and the property must contain one or more enumValues. Each of the enumValues must have a mandatory value element. The label element is optional.
- If the defaultValue is provided, it should match the value field of one of the enumValues.
- usersOnly and teamsOnly are mutually exclusive and only valid for USER properties
- defaultValue, if present, must match the correct type indicated by valueType
 - STRING: string
 - BOOL: boolean
 - INT: integer
 - DOUBLE: decimal
 - USER: string[]

- DATE: ISO date string
- BLOB and OBJECT are currently not supported
- USER-type property values are a list of user, team, and/or role IDs

States

Format

- name: string
- displayName: string
- approverSourceProperty?: string
- notifierSourceProperty?: string
- entryActions?: Action[]
- exitActions?: Action[]
- editableProperties?: string[]
 - Values may include:
 - approvers
 - observers
- requiredProperties?: string[]
- requiredItemProperties?: string[]

Restrictions

- Names must be unique
- approverSourceProperty and notifierSourceProperty, if present, must be the property IDs of the workflow and must point to USER-type properties
- At most, one state may use a given property as its approvers.
- Any number of states may use a given property as their notifiers.
- editableProperties and requiredProperties must be property IDs in the same workflow.
- requiredItemProperties must be valid metadata property IDs for your company (Onshape built-in properties or custom properties, excluding computed properties).



For more information on IDs, see [Understanding and Administering Project Roles and Permission Schemes](#).

Transitions

Format

- name: string
- displayName: string
- type: string
- sourceState: string
- targetState: string
- uiHint?: string (default: "primary")
- actions?: Action[]
- requiredProperties?: string[]

Restrictions

- Names must be unique.
- Type must be one of "APPROVE," "REJECT," or "SUBMIT".
- sourceState and targetState must be two different states within the workflow.
- There may be, at most, one APPROVE-type transition from any source state.
- For an APPROVE-type transition, sourceState must have an approverSourceProperty.
- uiHint, if present, must be either "primary," "success," or "danger" (these are the colors of the boxes and other entities in the release dialog and are hard-coded by Onshape).

Transition types

Submit

- Performs various "initialization" on the release as it goes out of set up and into the workflow, such as kicking off thumbnail generation for configured parts in the release, connecting linked document/version IDs, etc.
- Only the release creator has the ability to execute SUBMIT transitions.
- Transitions out of the initial state should be Submits.
- A Submit that isn't an initial transition does not have any special abilities

Approve

- Marks the user as having approved (turns the token green in the release dialog).
- If company policies are set to *Require all approvers*, the transition will not be performed until all approvers have approved.
- Generates assembly/drawing content references for items.
- Only one transition of this type is allowed out of a state.
- Only an approver of the current state (or an admin) has the ability to approve.

Reject

- Marks the user as having rejected (turns the token red in the release dialog).
- Only an approver of the current state (or an admin) has the ability to reject.

Actions

Format

- name: string
- params?: {[key: string]: any}

Restrictions

- Name must match one of our predefined action names (see below, under *Allowed Actions*)
- Some actions may be allowed only on states or on transitions, some may have ordering restrictions.

Allowed Actions

- **sendEmailNotifications** - Send email notifications for a transition (including mobile push notifications).
 - Allowed on: transitions only
 - Parameters: none
- **sendUserNotifications** - Send user notifications for a transition.
 - Allowed on: transitions only
 - Parameters: none

- **markItemsPending** - Change the metadata state of all items in the release to Pending.
 - Allowed on: states or transitions
 - Parameters: none
 - Not allowed on obsolescence workflows.
 - In release workflows, not possible after markItemsReleased or markItemsRejected.
- **markItemsRejected** - Change the metadata state of all items in the release to Rejected.
 - Allowed on: states or transitions
 - Parameters: none
 - Not allowed in obsolescence workflows.
 - In release workflows, not possible after markItemsReleased or before markItemsPending
 - **releaseItems** - Change the metadata state of all items in the release to Released and create revisions for them (auto-obsoleting other revisions if specified in company policy).
 Allowed on: states or transitions
 Parameters: none
 Not allowed on obsolescence workflows.
- **obsoleteItems** - Change the metadata state of all items in the package to Obsolete and obsolete their corresponding revisions.
 - Allowed on: states or transitions
 - Parameters: none
 - Not allowed on release workflows
- **updateItemPropertiesFromFeatureScript** - Use this to use a FeatureScript feature as a transition action, or as an entry or exit action on a state.
 - Allowed on: states or transitions
 - Parameters: map (single argument with this type)

Overriding company release settings

When defining a custom workflow, you can include an options section designed to override specific company release settings defined on the [Enterprise settings > Release management](#) page. These options for overriding are explained below:

```
"options": {
  "revisionSchemeId": string,
  "requireApprover": boolean,
  "requireAllApprovers": boolean,
  "disallowCreatorAsApprover": boolean,
  "requireNote": boolean,
  "autoObsolete": boolean,
  "errorOnFeatureListErrors": boolean,
  "errorOnRolledBack": boolean,
  "errorOnAssemblyErrors": boolean,
  "errorOnAssemblyRefsOutOfDate": boolean,
  "errorOnDrawingOutOfDate": boolean,
  "errorOnPartNumberPending": boolean,
  "errorOnPendingTask": boolean,
  "nameOverride": string, (overrides the Release name label, or Obsolescence name, in the Release dialog)
  "descriptionOverride": (overrides the Release notes label, or Obsolescence notes, in the Release dialog)
```

```
}
```

This section and all fields are optional. You can pick any subset with which you want to override a company setting. Any fields not included in the code will default to the specification in the Enterprise settings.

The revision Scheme Id can be retrieved from the read-only text box on the Enterprise settings > Release Management page:

Revisions and Part Numbers

Revision scheme:

Alphabetical A B C skipping I O Q S X Z

Revision scheme id: 583f626cc4236c12a9ceefb7

Copy to clipboard

Part number generation:

Sequential part number generation

Workflow Audit logging

All workflow objects create an audit trail of significant events in their lifetime, for example, creation and transition. This log can be retrieved using

GET /api/workflow/obj/<object id>/auditlog

You can view the output of that call through a link in the Release, in order to inspect and debug issues.

The audit log response contains basic information about the object workflow (including its published workflow version and whether it is a debug-mode workflow) and all events that are logged for it.

Each log entry contains:

- A timestamp
- A type (represented by an integer that is a BTWorkflowAuditEntryType ordinal)
- The workflow state/transition/action at the time of the event.

There are also some other fields for additional event-type-specific information, as applicable.

The types of audit events are:

- **Object created** - First event logged for an object, when it is created.
- **Object discarded** - When the special Discard action is performed on the object.
- **Object deleted** - When the object is removed, usually due to being old enough to be caught by the cleanup server (the audit log for a deleted object is not deleted).
- **Properties updated** - When workflow properties are changed on the object. This is for package-level properties on a release package (for example), not metadata properties on individual items.
 - This includes the property IDs (from the workflow definition) that were changed, including their old and new values.
- **Comment added** - Includes the ID of the newly added comment.
- **User approved** - When a user performs the approval action for the current state. This may or may not be accompanied by a transition, depending on the company release settings.
 - This includes the user/team/role IDs in the approver list for which the action was performed, that is, on whose behalf the user approved.
- **User rejected** - When a user performs the reject action for the current state.
 - This includes the user/team/role IDs in the approver list for which the action was performed, that is, on whose behalf the user rejected.
- **Transition started**
- **Transition completed**
- **Transition failed** - This includes an error message or support code, if applicable.
- **Action started**
- **Action completed**
- **Action failed** - This includes an error message or support code, if applicable.
- **FeatureScript executed** - This includes any console output from the FeatureScript execution.

- This includes any notices returned by the FeatureScript execution (including an exception, if one occurred).
 - Includes the value returned from the FeatureScript function.
 - **Item metadata updates failed** - When a release package item metadata update request fails during a transition.
 - This includes the failed metadata property IDs, the metadata hrefs of the failed items, and an error message.
 - Usually results in a transition failure, except when it happens during the FeatureScript action. Built-in actions need to succeed, but setting additional properties from FeatureScript can fail while allowing the transition to continue.
-

Last Updated: January 30, 2026